Universidad Tecnologica de Honduras
Ingenieria en Computacion

Creación de una API REST básica con PHP

Grupo 5

Fernando Abad Diaz Castro - 202410040257
Michael Abad Diaz Castro - 202310040165

Curso: Programación Móvil 1
Docente: Ing. Ricardo Lagos
Fecha: 31 de octubre de 2026

1. Introducción y objetivo

Las aplicaciones móviles modernas rara vez trabajan de forma aislada: casi todas obtienen y guardan su información en un servidor, y se comunican con él mediante una API. Este informe documenta la construcción de una API REST básica con PHP y MySQL que permite administrar un catálogo de productos, su validación mediante Postman y el proceso que seguiría una aplicación Android desarrollada con Java y XML para consumirla.

Objetivo general: crear una API REST básica en PHP que ejecute operaciones CRUD sobre el recurso "productos", probarla con Postman y explicar cómo una aplicación Android se conecta a ella.

Objetivos específicos:

- Explicar cómo PHP recibe peticiones HTTP y responde en formato JSON.
- Implementar los endpoints para listar, consultar, insertar, actualizar y eliminar productos.
- Aplicar validación de datos, manejo de errores y códigos de estado HTTP correctos.
- Probar y documentar cada endpoint con Postman, incluyendo casos de error.
- Describir el flujo de comunicación entre una aplicación Android y la API.

2. Marco teórico

2.1 PHP y su uso para crear una API

PHP es un lenguaje de programación del lado del servidor. Cuando un cliente (un navegador, Postman o una aplicación Android) envía una petición HTTP, el servidor web (Apache) la entrega a PHP, que la procesa y devuelve una respuesta. Para construir una API, PHP ofrece lo necesario de forma nativa:

- **$_SERVER['REQUEST_METHOD']:** indica el método HTTP de la petición (GET, POST, PUT, DELETE).
- **$_GET:** contiene los parámetros de la URL, por ejemplo ?id=1.
- **php://input:** permite leer el cuerpo (body) de la petición, donde llega el JSON enviado.
- **json_decode() y json_encode():** convierten entre texto JSON y estructuras de PHP.
- **header() y http_response_code():** definen los encabezados y el código de estado de la respuesta.
- **PDO:** extensión para conectarse a bases de datos de forma segura mediante consultas preparadas.

2.2 API y REST

Una API (Application Programming Interface) es un conjunto de reglas que permite que dos programas se comuniquen. REST (Representational State Transfer) es un estilo de arquitectura para diseñar APIs sobre HTTP. Sus principios básicos son:

- **Cliente-servidor:** el cliente (app Android) y el servidor (PHP + MySQL) son independientes.

- **Sin estado:** cada petición contiene toda la información necesaria; el servidor no recuerda peticiones anteriores.
- **Recursos con URL:** cada recurso (por ejemplo, un producto) se identifica con una URL, como productos.php?id=1.
- **Interfaz uniforme:** se usan los métodos HTTP con su significado estándar (GET lee, POST crea, PUT actualiza, DELETE elimina).
- **Representaciones:** los datos se intercambian en un formato común, normalmente JSON.

Endpoint: es la URL concreta donde la API atiende un tipo de petición. Ejemplo: http://localhost:8080/productos.php?id=1.

2.3 Formato JSON

JSON (JavaScript Object Notation) es un formato de texto ligero para intercambiar datos, formado por pares clave-valor. Es legible para personas y fácil de procesar para las máquinas, por lo que es el formato estándar entre una API y una app Android. Ejemplo de un producto:

```
{
  "id": 1,
  "nombre": "Mouse inalámbrico",
  "descripcion": "Mouse óptico 2.4 GHz",
  "precio": 250,
  "stock": 30
}
```

2.4 Métodos HTTP

Método	Función	Uso en esta API
GET	Obtener información sin modificarla	Listar productos y consultar uno por id
POST	Crear un nuevo recurso	Insertar un producto
PUT	Reemplazar/actualizar un recurso	Actualizar un producto por id
PATCH	Actualizar parcialmente un recurso	No implementado (responde 405)
DELETE	Eliminar un recurso	Eliminar un producto por id

2.5 Códigos de estado HTTP

Código	Significado	Cuándo lo devuelve la API
200 OK	La petición fue exitosa	Listar, consultar, actualizar y eliminar
201 Created	El recurso fue creado	Al insertar un producto con POST
400 Bad Request	La petición es incorrecta	JSON inválido, datos que no pasan la validación, id no numérico
401 Unauthorized	Falta autenticación	No aplica (la API básica no tiene login)

Código	Significado	Cuándo lo devuelve la API
404 Not Found	El recurso no existe	Se consulta, actualiza o elimina un id inexistente
405 Method Not Allowed	Método no permitido	Se usa un método no soportado, como PATCH
500 Internal Server Error	Falla interna del servidor	No se puede conectar a la base de datos o falla una consulta

2.6 Flujo general de comunicación

Aplicación Android → petición HTTP → API REST (PHP) → procesamiento (validación y consulta a MySQL) → respuesta JSON → visualización en la aplicación.

3. Diseño de la API

El recurso elegido es "productos". Todos los endpoints se atienden desde un único archivo, productos.php, que decide qué hacer según el método HTTP recibido. La URL base es http://localhost:8080/ (en un emulador Android se usa http://10.0.2.2:8080/).

Operación	Método y URL	Datos enviados	Respuesta esperada
Listar todos	GET productos.php	Ninguno	200 + arreglo JSON con todos los productos
Consultar uno	GET productos.php?id=1	id en la URL	200 + objeto del producto 404 si no existe
Insertar	POST productos.php	JSON: nombre, descripcion, precio, stock	201 + producto creado 400 si los datos son inválidos
Actualizar	PUT productos.php?id=1	id en la URL y JSON completo	200 + producto actualizado 400 / 404
Eliminar	DELETE productos.php?id=1	id en la URL	200 + mensaje de confirmación 404 si no existe

Reglas de validación: nombre obligatorio (texto, máximo 100 caracteres); precio obligatorio (número mayor o igual a 0); stock obligatorio (entero mayor o igual a 0); descripcion opcional (texto).

4. Base de datos

Se usa MySQL 8, ejecutado en un contenedor de Docker. El siguiente script crea la tabla productos y tres registros de ejemplo; Docker lo ejecuta automáticamente la primera vez que arranca el contenedor de MySQL (archivo init.sql). También puede ejecutarse manualmente en phpMyAdmin, en la pestaña SQL.

```
CREATE DATABASE IF NOT EXISTS api_productos
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE api_productos;
```

```
CREATE TABLE IF NOT EXISTS productos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  descripcion VARCHAR(255) DEFAULT NULL,
  precio DECIMAL(10,2) NOT NULL,
  stock INT NOT NULL DEFAULT 0,
  creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO productos (nombre, descripcion, precio, stock) VALUES
('Mouse inalámbrico', 'Mouse óptico 2.4 GHz', 250.00, 30),
('Teclado mecánico', 'Teclado con switches azules', 950.50, 15),
('Audífonos', 'Audífonos con micrófono', 420.00, 25);
```

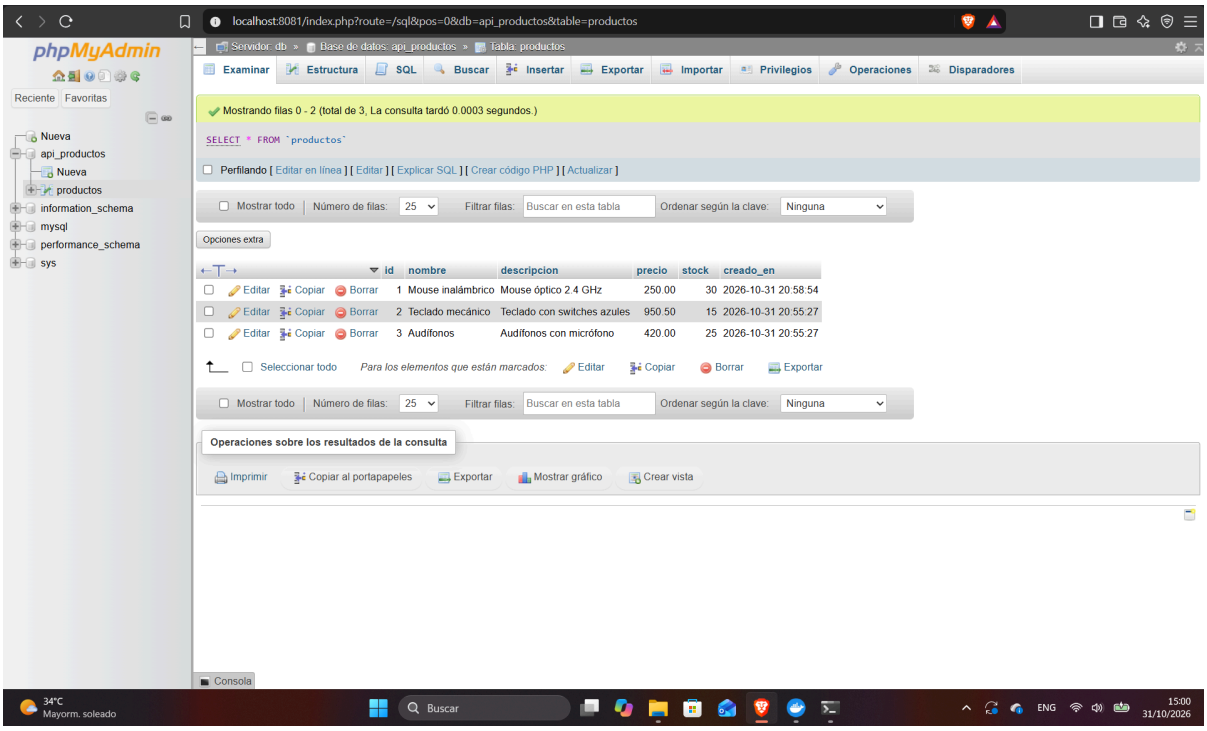


Figura 1. Tabla productos en phpMyAdmin

5. Implementación y explicación del código

El proyecto está formado por dos archivos PHP (conexion.php y productos.php) dentro de la carpeta src, más los archivos de Docker (Dockerfile, docker-compose.yml e init.sql). El código completo de los archivos PHP está en el Anexo A; aquí se explica la lógica de cada parte.

5.1 conexion.php

- **Configuración:** define servidor, base de datos, usuario y contraseña. Dentro de Docker las toma de las variables de entorno definidas en docker-compose.yml; si no existen, usa valores por defecto para ejecutarse también en XAMPP (usuario root sin contraseña).

- **Encabezados HTTP:** Content-Type: application/json hace que toda respuesta se entregue como JSON en UTF-8. Los encabezados Access-Control-Allow-* habilitan CORS para permitir peticiones desde otros orígenes.
- **Petición OPTIONS:** algunos clientes envían primero una petición de verificación (preflight); la API responde 204 y termina.
- **Función responder():** centraliza la salida: fija el código de estado, imprime el JSON y detiene la ejecución. Así todas las respuestas tienen el mismo formato.
- **Conexión PDO:** se abre con ERRMODE_EXCEPTION para capturar errores con try/catch. Si la conexión falla, se responde 500 con un mensaje genérico, sin exponer datos internos del servidor.

5.2 productos.php

- **Recepción de la petición:** lee el método con $_SERVER['REQUEST_METHOD'] y el id con $_GET['id']. Si el id no es un entero positivo (ctype_digit), responde 400.
- **formatear():** MySQL devuelve todos los valores como texto; esta función convierte id y stock a entero y precio a decimal para que el JSON tenga los tipos correctos, algo importante para que Gson/Retrofit lo interprete bien en Android.
- **buscar():** consulta un producto por id con una consulta preparada y devuelve null si no existe, lo que permite responder 404.
- **leerBody():** lee php://input y lo decodifica con json_decode. Si el texto no es un JSON válido, responde 400.
- **validar():** revisa cada campo y devuelve una lista con todos los errores encontrados, de modo que el cliente pueda corregirlos de una vez.
- **switch según el método:** GET lista o consulta, POST inserta y devuelve 201 con el producto creado, PUT actualiza (verifica que exista, valida y actualiza), DELETE elimina (verifica que exista). Cualquier otro método devuelve 405.
- **Manejo de errores:** todo el bloque está dentro de try/catch; ante un error de base de datos se responde 500 con un mensaje genérico.

5.3 Seguridad: consultas preparadas

Todas las consultas usan marcadores de posición (?) y execute() con los valores por separado, en lugar de concatenar texto en el SQL. Esto evita la inyección SQL, una de las vulnerabilidades más comunes en aplicaciones web.

```
$st = $pdo->prepare('SELECT * FROM productos WHERE id = ?');
$st->execute([$id]);
```

5.4 Cómo ponerla en marcha

- Instalar Docker Desktop y abrirlo.
- Descomprimir la carpeta del proyecto y abrir una terminal dentro de ella.
- Ejecutar docker compose up -d --build. Este comando construye la imagen de PHP con Apache, inicia MySQL y phpMyAdmin, y crea la tabla productos con datos de ejemplo.
- Probar en el navegador: http://localhost:8080/productos.php debe mostrar el JSON de los productos. phpMyAdmin queda disponible en http://localhost:8081.

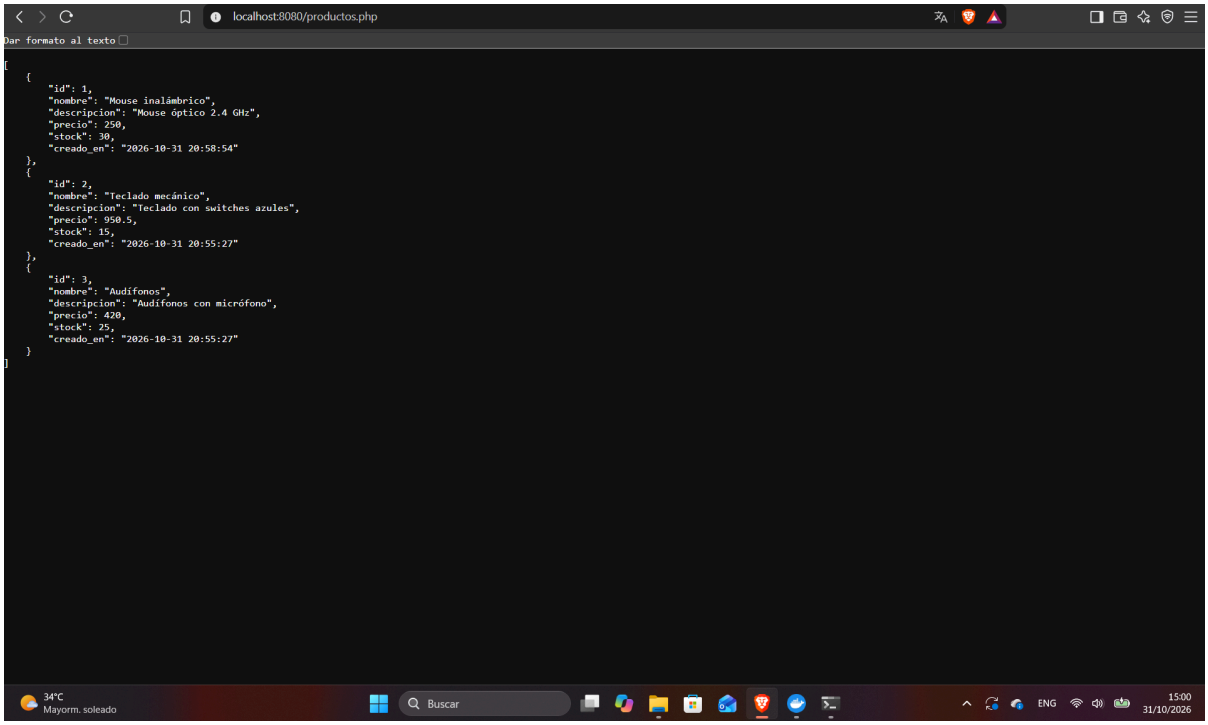


Figura 2. La API respondiendo desde el navegador

6. Pruebas con Postman

Se creó una colección llamada "API Productos PHP" con las 10 peticiones de prueba. Postman permite elegir el método, escribir la URL, enviar el cuerpo en formato JSON (pestaña Body → raw → JSON) y revisar el código de estado y la respuesta. Antes de conectar una app Android conviene validar la API con Postman, porque así se separan los errores del servidor de los errores de la aplicación. En el archivo API_Productos_PHP.postman_collection.json se entrega la colección lista para importar (Import → seleccionar el archivo), con una variable base_url que se puede cambiar según el entorno.

6.1 Pruebas de funcionamiento (CRUD)

Prueba 1: Listar todos

Petición	Body	Código esperado
`GET http://localhost:8080/productos.php`	–	200

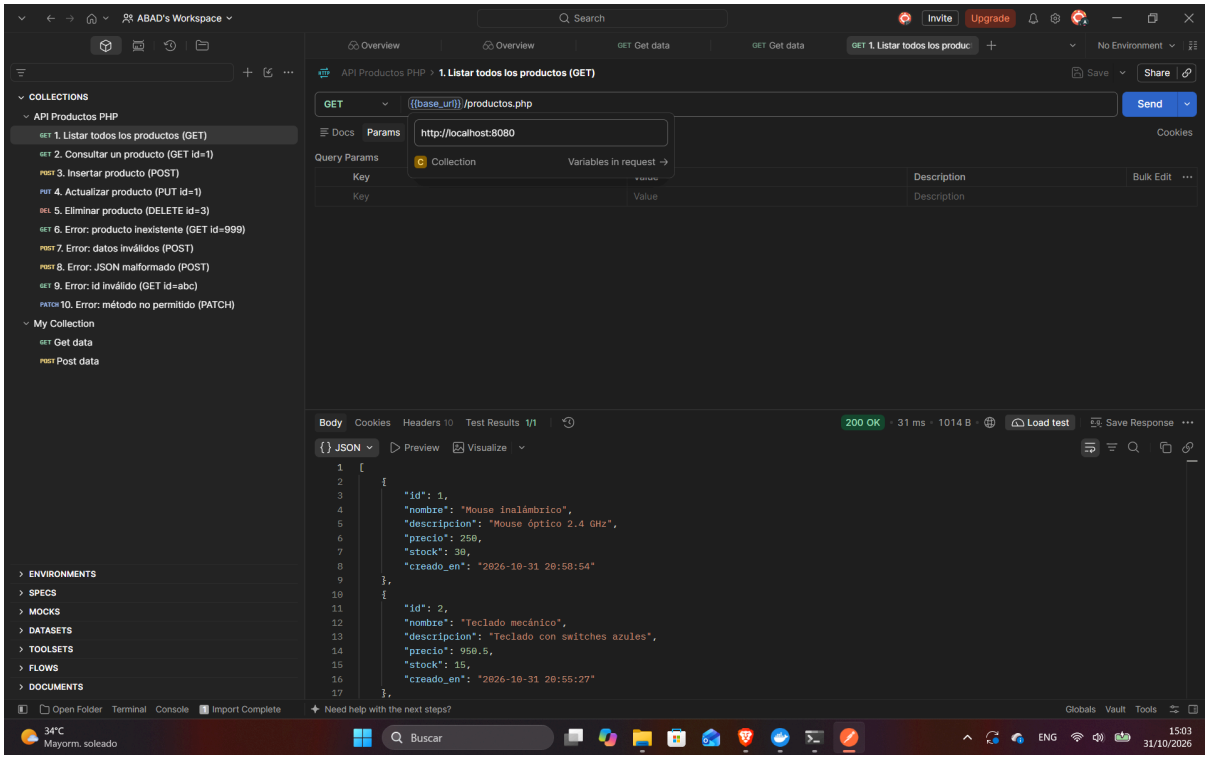


Figura 3. Prueba 1: Listar todos

Se envió una petición GET a productos.php sin parámetros. La API respondió 200 OK con un arreglo JSON que contiene los tres productos registrados.

Prueba 2: Consultar uno

Petición	Body	Código esperado
`GET .../productos.php?id=1`	–	200

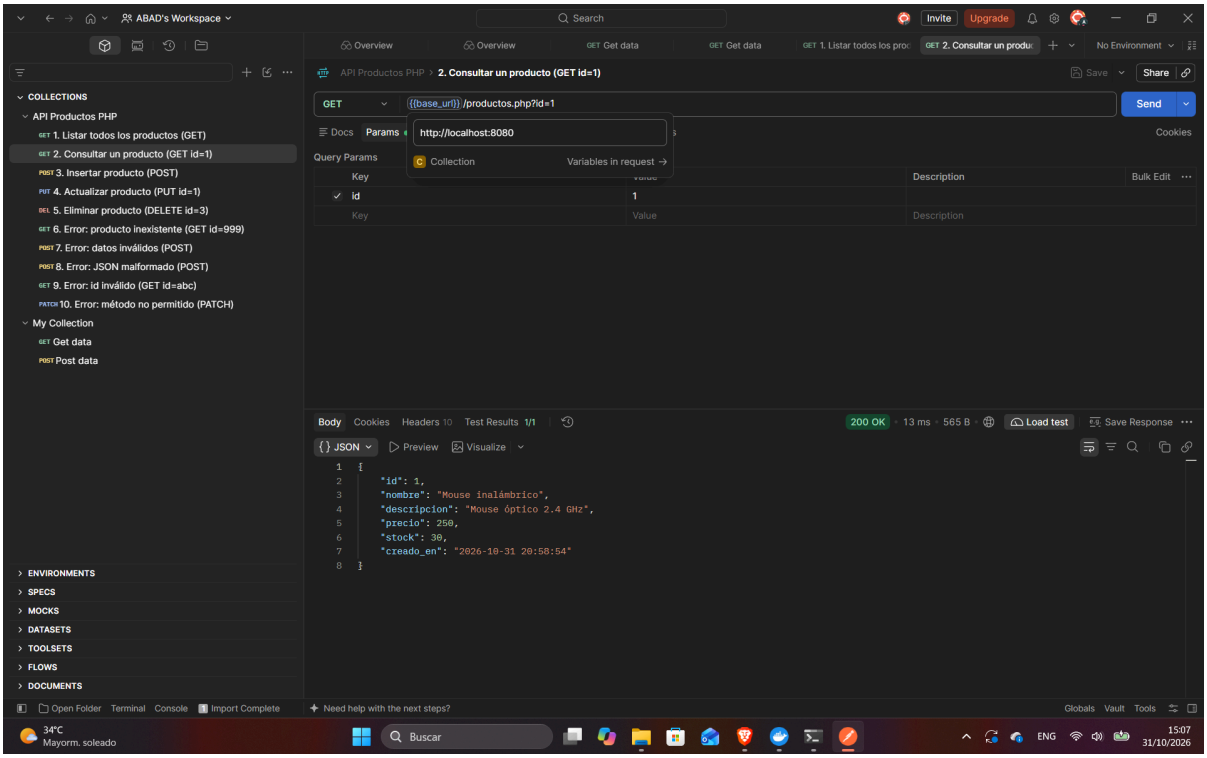


Figura 4. Prueba 2: Consultar uno

Se envió una petición GET con el parámetro id=1. La API respondió 200 OK con el objeto JSON del producto "Mouse inalámbrico".

Prueba 3: Insertar

Petición	Body	Código esperado
`POST .../productos.php`	`{"nombre":"Monitor 24\"","descripcion":"Full HD","precio":3200.00,"stock":8}`	201

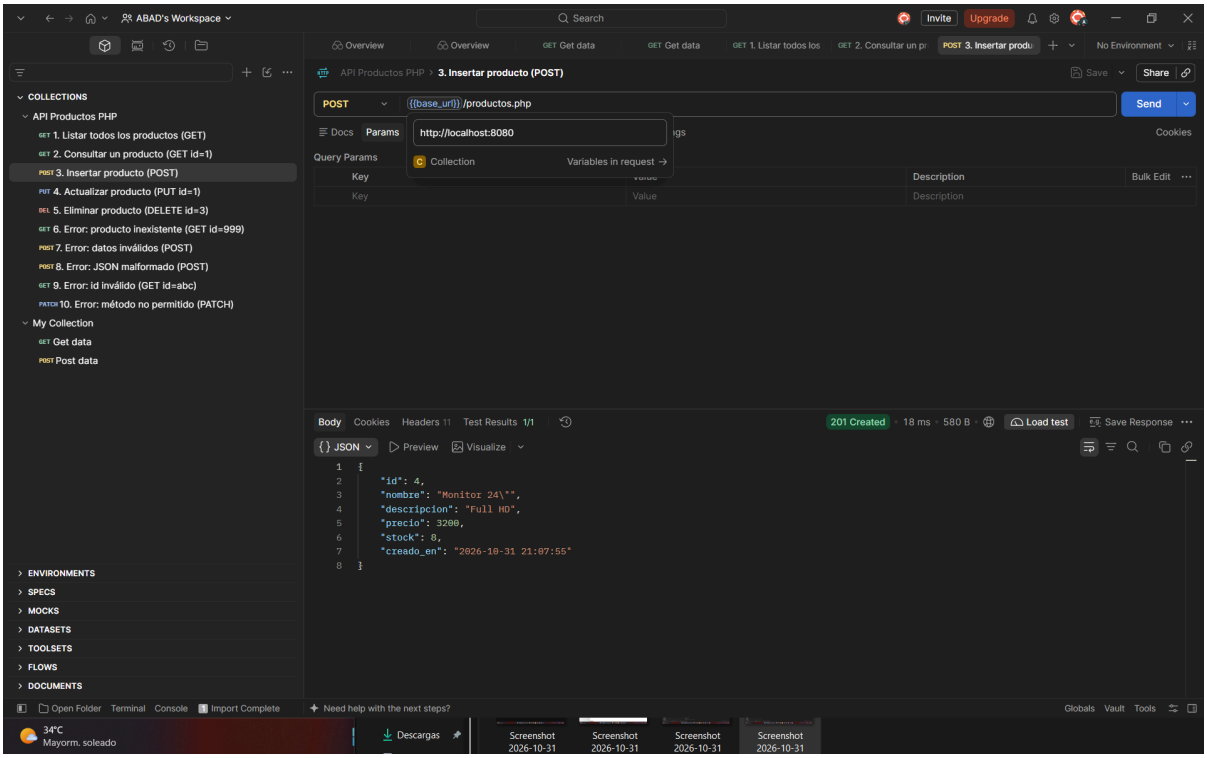


Figura 5. Prueba 3: Insertar

Se envió una petición POST con un JSON que contiene nombre, descripcion, precio y stock. La API validó los datos, insertó el producto y respondió 201 Created con el producto creado y su nuevo id.

Prueba 4: Actualizar

Petición	Body	Código esperado
`PUT .../productos.php?id=1`	`{"nombre":"Mouse gamer","descripcion":"RGB","precio":399.99,"stock":20}`	200

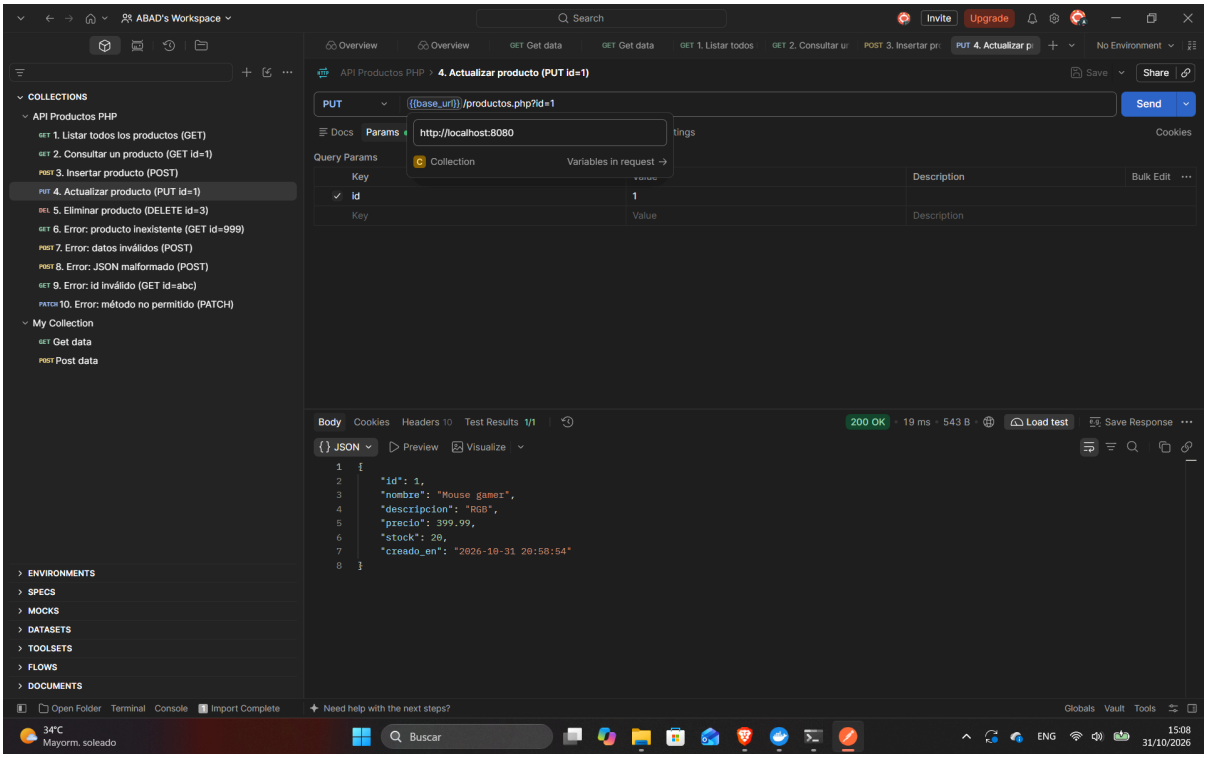


Figura 6. Prueba 4: Actualizar

Se envió una petición PUT con id=1 y un JSON con los datos nuevos. La API actualizó el registro y respondió 200 OK con el producto modificado ("Mouse gamer").

Prueba 5: Eliminar

Petición	Body	Código esperado
`DELETE .../productos.php?id=3`	–	200

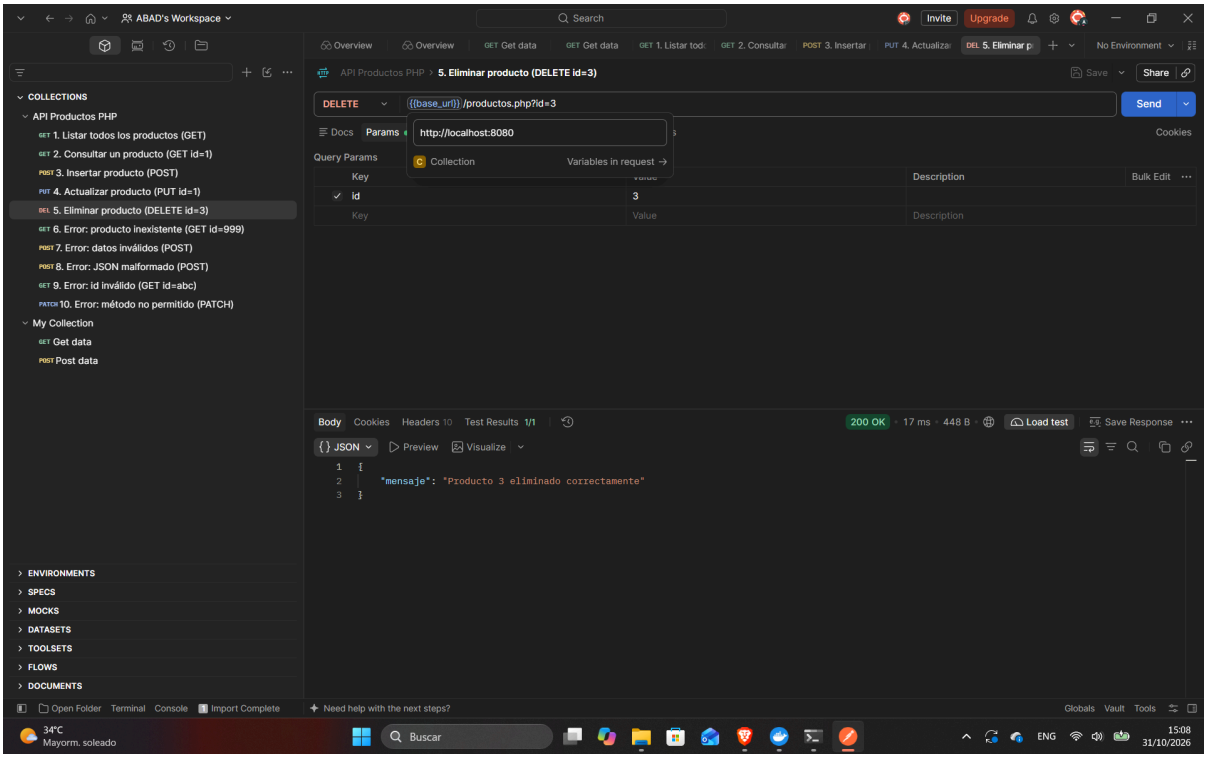


Figura 7. Prueba 5: Eliminar

Se envió una petición DELETE con id=3. La API eliminó el producto y respondió 200 OK con un mensaje de confirmación.

6.2 Pruebas de manejo de errores

Prueba 6: Producto inexistente

Petición	Body	Código esperado
`GET .../productos.php?id=999`	–	404

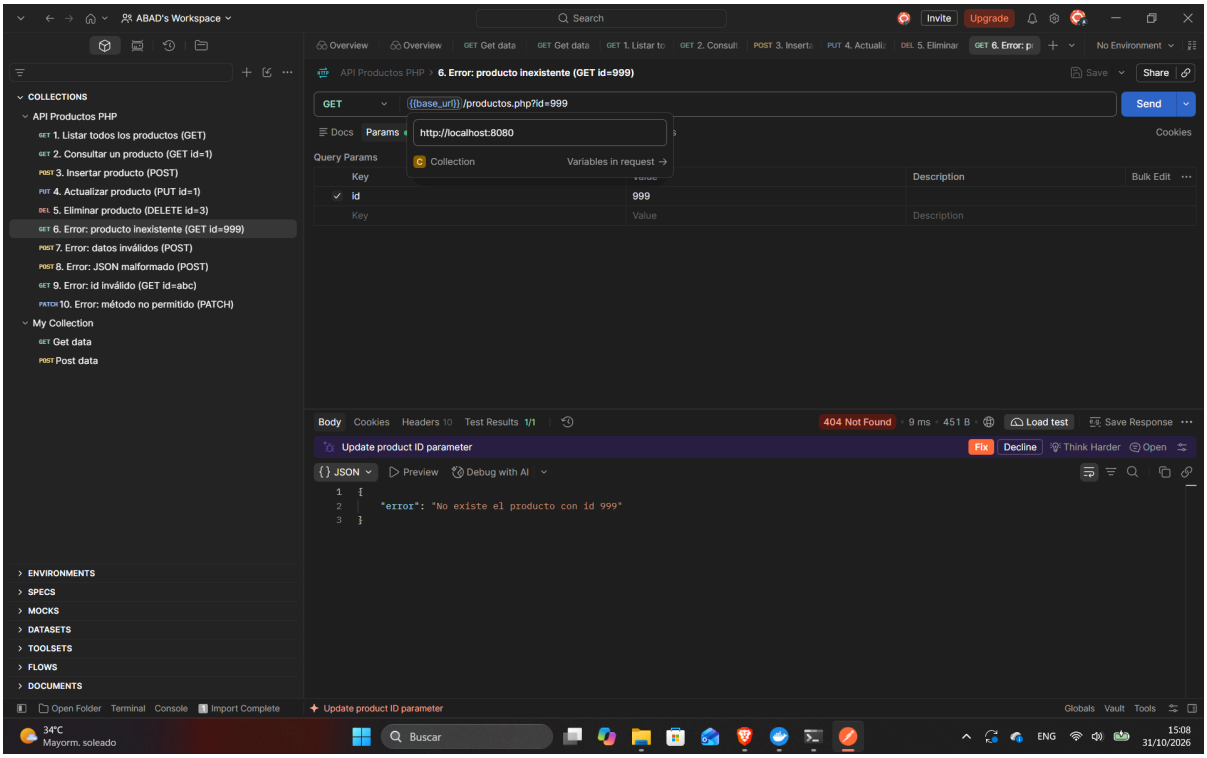


Figura 8. Prueba 6: Producto inexistente

Se consultó el id 999, que no existe en la base de datos. La API respondió 404 Not Found con el mensaje "No existe el producto con id 999".

Prueba 7: Datos inválidos

Petición	Body	Código esperado
`POST .../productos.php`	`{"nombre":"","precio":-5}`	400

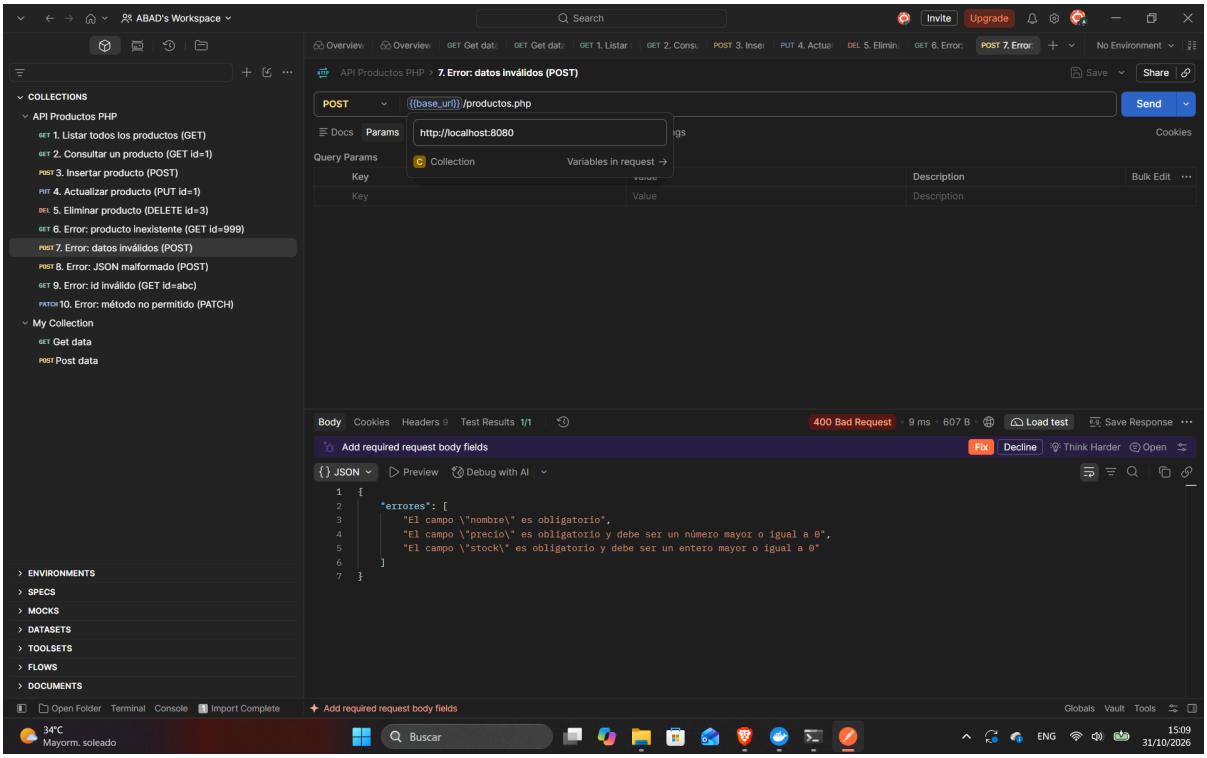


Figura 9. Prueba 7: Datos inválidos

Se envió un POST con el nombre vacío, un precio negativo y sin stock. La API respondió 400 Bad Request con la lista de errores de validación de cada campo.

Prueba 8: JSON malformado

Petición	Body	Código esperado
`POST .../productos.php`	`hola`	400

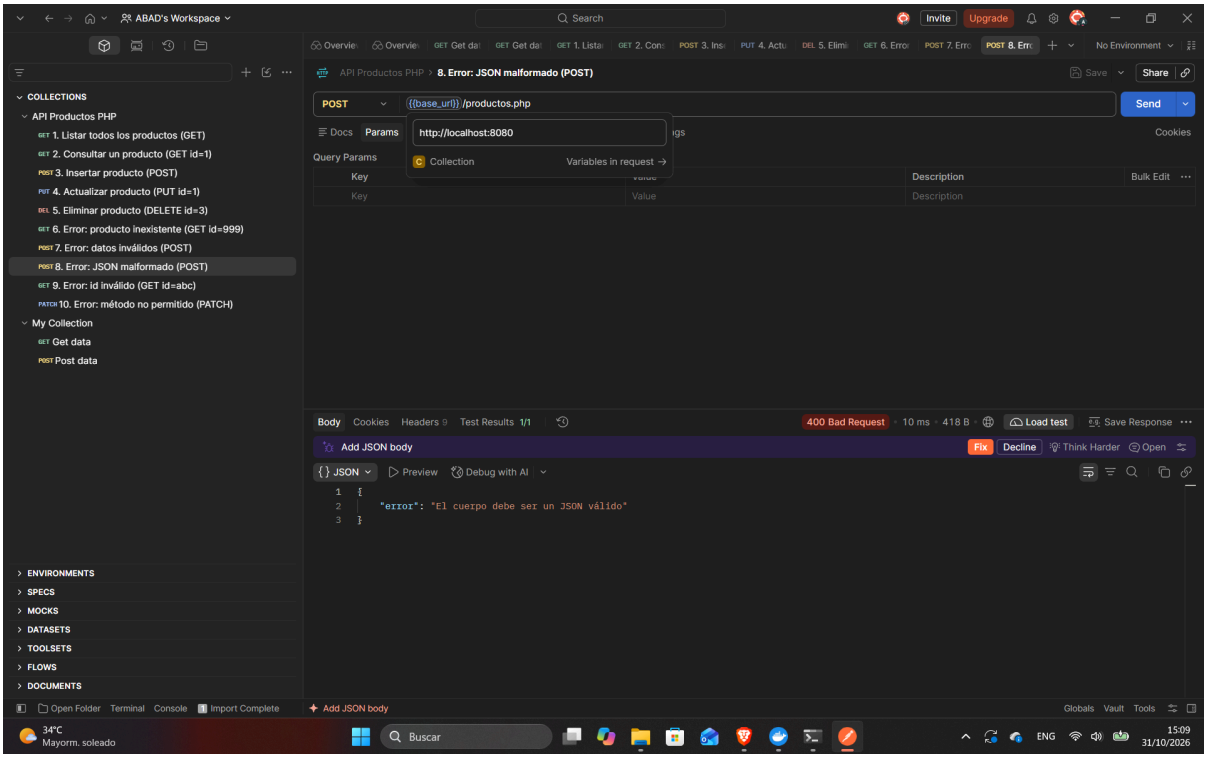


Figura 10. Prueba 8: JSON malformado

Se envió un POST con el texto "hola" como cuerpo, que no es un JSON válido. La API respondió 400 Bad Request indicando que el cuerpo debe ser un JSON válido.

Prueba 9: Id inválido

Petición	Body	Código esperado
`GET .../productos.php?id=abc`	–	400

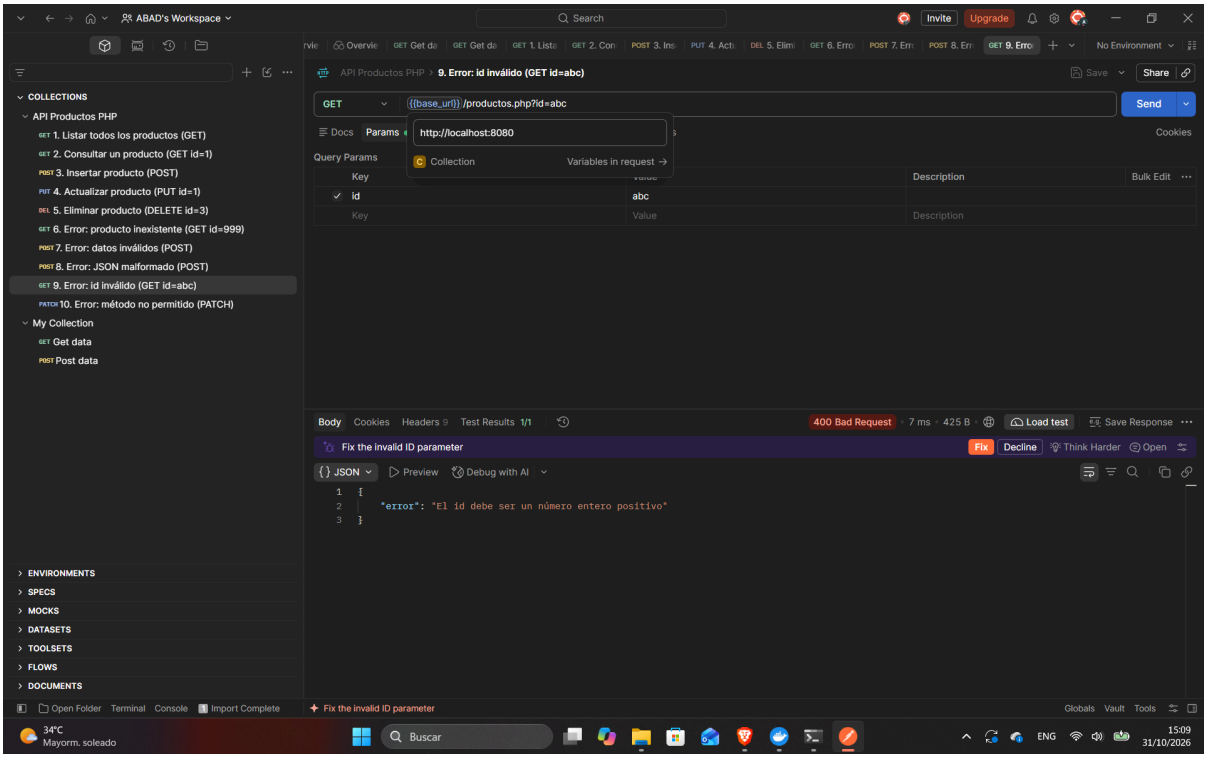


Figura 11. Prueba 9: Id inválido

Se consultó con id=abc, que no es un número. La API respondió 400 Bad Request indicando que el id debe ser un número entero positivo.

Prueba 10: Método no permitido

Petición	Body	Código esperado
`PATCH .../productos.php`	–	405

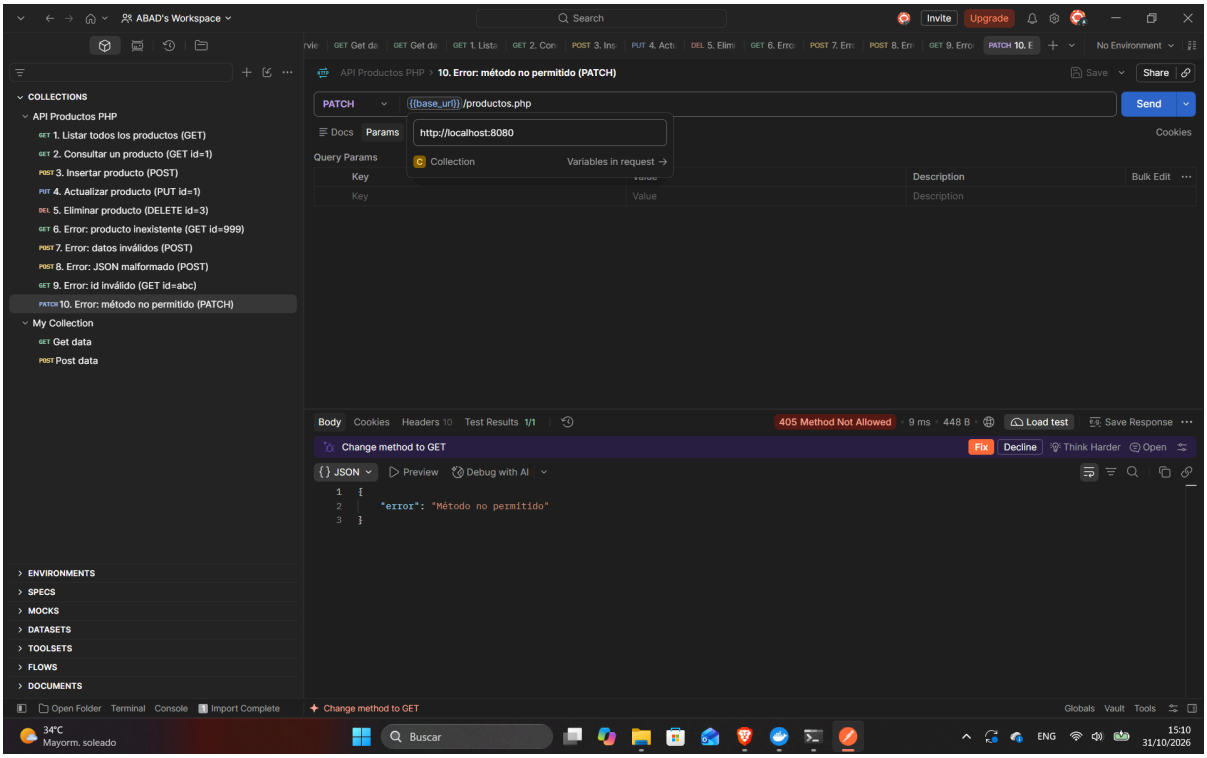


Figura 12. Prueba 10: Método no permitido

Se usó el método PATCH, que la API no implementa. La API respondió 405 Method Not Allowed con el mensaje "Método no permitido".

6.3 Resumen de resultados

#	Prueba	Esperado	Obtenido
1	Listar todos	200	200 OK
2	Consultar uno	200	200 OK
3	Insertar	201	201 Created
4	Actualizar	200	200 OK
5	Eliminar	200	200 OK
6	Producto inexistente	404	404 Not Found
7	Datos inválidos	400	400 Bad Request
8	JSON malformado	400	400 Bad Request
9	Id inválido	400	400 Bad Request
10	Método no permitido	405	405 Method Not Allowed

7. Conexión desde una aplicación Android

El flujo es:

Aplicación Android (Activity + Retrofit) → petición HTTP → productos.php → procesamiento (validación y consulta a MySQL) → respuesta JSON → conversión a objetos Java con Gson → visualización en un RecyclerView.

7.1 Configuración del proyecto

Se agrega Retrofit con el convertidor Gson en el archivo build.gradle del módulo app (conviene verificar la versión más reciente en el sitio oficial de Retrofit):

```
dependencies {
    implementation 'com.squareup.retrofit2:retrofit:2.9.0'
    implementation 'com.squareup.retrofit2:converter-gson:2.9.0'
}
```

En AndroidManifest.xml se declara el permiso de Internet y, como el servidor local usa HTTP y no HTTPS, se permite el tráfico sin cifrar durante el desarrollo:

```
<uses-permission android:name="android.permission.INTERNET"/>

<application
    android:usesCleartextTraffic="true"
    ... >
```

7.2 Modelo (POJO)

La clase Producto representa el JSON; los nombres de los atributos coinciden con las claves devueltas por la API.

```
public class Producto {
    private int id;
    private String nombre;
    private String descripcion;
    private double precio;
    private int stock;
    // getters y setters
}
```

7.3 Interfaz con los endpoints

```
public interface ApiProductos {
    @GET("productos.php")
    Call<List<Producto>> listar();

    @GET("productos.php")
    Call<Producto> obtener(@Query("id") int id);

    @POST("productos.php")
    Call<Producto> crear(@Body Producto producto);

    @PUT("productos.php")
```

```
    Call<Producto> actualizar(@Query("id") int id, @Body Producto producto);

    @DELETE("productos.php")
    Call<ResponseBody> eliminar(@Query("id") int id);
}
```

Las anotaciones @GET, @POST, @PUT y @DELETE indican el método HTTP; @Query agrega el parámetro ?id= a la URL y @Body convierte el objeto Java a JSON automáticamente.

7.4 Cliente de Retrofit

```
Retrofit retrofit = new Retrofit.Builder()
    .baseUrl("http://10.0.2.2:8080/")  // 10.0.2.2 = localhost de la PC visto desde el emulador
    .addConverterFactory(GsonConverterFactory.create())
    .build();
ApiProductos api = retrofit.create(ApiProductos.class);
```

En un teléfono físico se usa la dirección IP de la computadora en la red local (por ejemplo http://192.168.1.10:8080/), y ambos dispositivos deben estar en la misma red.

7.5 Petición asíncrona y visualización

Las peticiones de red no pueden ejecutarse en el hilo principal. Retrofit lo resuelve con enqueue(), que ejecuta la llamada en segundo plano y devuelve el resultado en un callback:

```
api.listar().enqueue(new Callback<List<Producto>>() {
    @Override
    public void onResponse(Call<List<Producto>> call, Response<List<Producto>> response) {
        if (response.isSuccessful() && response.body() != null) {
            adapter.setDatos(response.body());   // actualiza el RecyclerView
        } else {
            Toast.makeText(MainActivity.this, "Error " + response.code(),
Toast.LENGTH_SHORT).show();
        }
    }

    @Override
    public void onFailure(Call<List<Producto>> call, Throwable t) {
        Toast.makeText(MainActivity.this, "Sin conexión: " + t.getMessage(),
Toast.LENGTH_LONG).show();
    }
});
```

La interfaz se define en XML con un RecyclerView que muestra la lista de productos:

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerProductos"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Los códigos de estado de la API se aprovechan en la app: onResponse con isSuccessful() cubre los códigos 2xx; los códigos 400 y 404 llegan también a onResponse con isSuccessful() en falso, y se pueden mostrar al usuario; onFailure se ejecuta cuando no hay conexión con el servidor.

8. Conclusiones

- PHP permite construir una API REST funcional con pocos archivos, usando únicamente sus funciones nativas y PDO para la base de datos.
- Separar la conexión (conexion.php) de la lógica de los endpoints (productos.php) y centralizar las respuestas en la función responder() hace el código más ordenado y fácil de mantener.
- Validar los datos recibidos y devolver el código de estado HTTP correcto (400, 404, 405, 500) permite que el cliente, sea Postman o una app Android, sepa exactamente qué ocurrió.
- Las consultas preparadas con PDO protegen la API contra la inyección SQL.
- Probar la API con Postman antes de conectarla a Android permite detectar y corregir errores del servidor de forma aislada.
- Una app Android con Java y XML puede consumir la API usando Retrofit, convirtiendo el JSON en objetos Java y mostrándolos en un RecyclerView mediante peticiones asíncronas.
- Como mejora futura, la API podría incorporar autenticación con tokens, soporte para PATCH, paginación y el uso de HTTPS en un servidor público.

9. Referencias

- PHP Group. Manual de PHP. https://www.php.net/manual/es/
- PHP Group. PHP Data Objects (PDO). https://www.php.net/manual/es/book.pdo.php
- Mozilla Developer Network. Métodos de petición HTTP. https://developer.mozilla.org/es/docs/Web/HTTP/Methods
- Mozilla Developer Network. Códigos de estado de respuesta HTTP. https://developer.mozilla.org/es/docs/Web/HTTP/Status
- JSON.org. Introducción a JSON. https://www.json.org/json-es.html
- Postman. Postman Learning Center. https://learning.postman.com/docs/
- Square. Retrofit. https://square.github.io/retrofit/
- Android Developers. Conectarse a la red. https://developer.android.com/training/basics/network-ops/connecting
- Docker Inc. Documentación de Docker y Docker Compose. https://docs.docker.com/

Anexo A. Código completo

A.1 conexion.php

```
<?php
// En Docker se leen de variables de entorno; en XAMPP se usan los valores por defecto.
$host = getenv('DB_HOST') ?: 'localhost';
$bd   = getenv('DB_NAME') ?: 'api_productos';
$user = getenv('DB_USER') ?: 'root';
$pass = getenv('DB_PASS') !== false ? getenv('DB_PASS') : '';
header('Content-Type: application/json; charset=utf-8');
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type, Authorization');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    http_response_code(204);
    exit;
}

function responder(int $codigo, $datos): void {
    http_response_code($codigo);
    echo json_encode($datos, JSON_UNESCAPED_UNICODE | JSON_PRETTY_PRINT);
    exit;
}

try {
    $pdo = new PDO(
        "mysql:host=$host;dbname=$bd;charset=utf8mb4",
        $user,
        $pass,
        [
            PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        ]
    );
} catch (PDOException $e) {
    responder(500, ['error' => 'No se pudo conectar a la base de datos']);
}
```

A.2 productos.php

```
<?php
require_once 'conexion.php';

$metodo = $_SERVER['REQUEST_METHOD'];
$id = $_GET['id'] ?? null;

// Si viene un id, debe ser un entero positivo
if ($id !== null && !ctype_digit((string)$id)) {
    responder(400, ['error' => 'El id debe ser un número entero positivo']);
}


// Convierte los tipos de la fila (MySQL devuelve todo como texto)
function formatear(array $p): array {
    return [
        'id'          => (int)$p['id'],
        'nombre'      => $p['nombre'],
        'descripcion' => $p['descripcion'],
        'precio'      => (float)$p['precio'],
```

```
        'stock'       => (int)$p['stock'],
        'creado_en'   => $p['creado_en'],
    ];
}

function buscar(PDO $pdo, int $id): ?array {
    $st = $pdo->prepare('SELECT * FROM productos WHERE id = ?');
    $st->execute([$id]);
    $fila = $st->fetch();
    return $fila ? formatear($fila) : null;
}


function leerBody(): array {
    $datos = json_decode(file_get_contents('php://input'), true);
    if (!is_array($datos)) {
        responder(400, ['error' => 'El cuerpo debe ser un JSON válido']);
    }
    return $datos;
}


function validar(array $d): array {
    $errores = [];

    if (!isset($d['nombre']) || !is_string($d['nombre']) || trim($d['nombre']) === '') {
        $errores[] = 'El campo "nombre" es obligatorio';
    } elseif (mb_strlen($d['nombre']) > 100) {
        $errores[] = 'El "nombre" no puede superar 100 caracteres';
    }

    if (!isset($d['precio']) || !is_numeric($d['precio']) || $d['precio'] < 0) {
        $errores[] = 'El campo "precio" es obligatorio y debe ser un número mayor o igual a 0';
    }

    if (!isset($d['stock']) || filter_var($d['stock'], FILTER_VALIDATE_INT) === false || $d['stock']
< 0) {
        $errores[] = 'El campo "stock" es obligatorio y debe ser un entero mayor o igual a 0';
    }

    if (isset($d['descripcion']) && !is_string($d['descripcion'])) {
        $errores[] = 'La "descripcion" debe ser texto';
    }

    return $errores;
}


try {
    switch ($metodo) {

        case 'GET':
            if ($id === null) {
                $filas = $pdo->query('SELECT * FROM productos ORDER BY id')->fetchAll();
                responder(200, array_map('formatear', $filas));
            }
            $producto = buscar($pdo, (int)$id);
            if (!$producto) {
                responder(404, ['error' => "No existe el producto con id $id"]);
            }
            responder(200, $producto);
            break;

        case 'POST':
```

```
            if ($id !== null) {
                responder(400, ['error' => 'No envíes id al crear un producto']);
            }
            $datos = leerBody();
            $errores = validar($datos);
            if ($errores) {
                responder(400, ['errores' => $errores]);
            }
            $st = $pdo->prepare(
                'INSERT INTO productos (nombre, descripcion, precio, stock) VALUES (?, ?, ?, ?)'
            );
            $st->execute([
                trim($datos['nombre']),
                $datos['descripcion'] ?? null,
                $datos['precio'],
                $datos['stock'],
            ]);
            $nuevo = buscar($pdo, (int)$pdo->lastInsertId());
            header('Location: productos.php?id=' . $nuevo['id']);
            responder(201, $nuevo);
            break;

        case 'PUT':
            if ($id === null) {
                responder(400, ['error' => 'Debes indicar el id: productos.php?id=1']);
            }
            if (!buscar($pdo, (int)$id)) {
                responder(404, ['error' => "No existe el producto con id $id"]);
            }
            $datos = leerBody();
            $errores = validar($datos);
            if ($errores) {
                responder(400, ['errores' => $errores]);
            }
            $st = $pdo->prepare(
                'UPDATE productos SET nombre = ?, descripcion = ?, precio = ?, stock = ? WHERE id =
?'
            );
            $st->execute([
                trim($datos['nombre']),
                $datos['descripcion'] ?? null,
                $datos['precio'],
                $datos['stock'],
                (int)$id,
            ]);
            responder(200, buscar($pdo, (int)$id));
            break;

        case 'DELETE':
            if ($id === null) {
                responder(400, ['error' => 'Debes indicar el id: productos.php?id=1']);
            }
            if (!buscar($pdo, (int)$id)) {
                responder(404, ['error' => "No existe el producto con id $id"]);
            }
            $st = $pdo->prepare('DELETE FROM productos WHERE id = ?');
            $st->execute([(int)$id]);
            responder(200, ['mensaje' => "Producto $id eliminado correctamente"]);
            break;

        default:
            responder(405, ['error' => 'Método no permitido']);
    }
} catch (PDOException $e) {
    responder(500, ['error' => 'Error interno del servidor']);
```

```
}
```